

Programming 2025/2026

Laboratory

Session 8 (May 8)

List, Dictionary and Tuple Comprehensions

First Cycle Degree/Bachelor in Genomics

Dept. Of Pharmacy and Biotechnology (FaBiT)

University of Bologna



List Comprehension

- List comprehension is a compact way to construct a list of elements following a specific rule

E.g., $\{x^2 | x \in [0, 4]\}$

```
square = [x**2 for x in range(5)]  
print(square)
```

[0, 1, 4, 9, 16]

- We can also exploit branching (`if`) in list comprehension

E.g., $\{x^2 | x \in [0, 4] \wedge x \text{ is even}\}$

```
squareven = [x**2 for x in range(5) if x%2 == 0]  
print(squareven)
```

[0, 4, 16]

List Comprehension

- A list comprehension consists of square brackets `[ ]` containing one `expression` followed by a `for` loop, then zero or more `for` loops and `conditions`

```
new_list = [expression for_loop+ conditions*]
```

`expression`: is the operation that we want to apply to a certain data

`for loop`: populates the list by applying a certain expression to the data

`conditions`: determine whether or not to apply the expression and include the data in the list

- The result will be a new list resulting from the evaluation of the expression in the context of the `for` and `if` that follows it

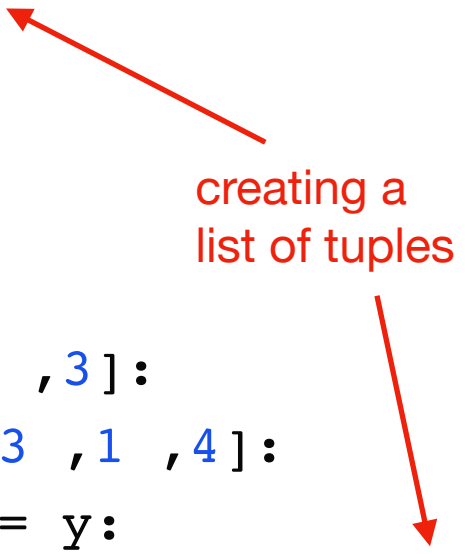
Examples

- The following comprehension combines the elements of two lists if they are not equal:

```
comb = [(x, y) for x in [1, 2, 3] for y in [3, 1, 4] if x != y]
```

- It is the same as:

```
comb = list()
for x in [1, 2, 3]:
    for y in [3, 1, 4]:
        if x != y:
            comb.append((x, y))
```



creating a
list of tuples

Examples

```
>> vec = [-4, -2, 0, 2, 4]
```

```
# create a new list with values of a list multiplied by 2
```

```
>> [x*2 for x in vec]
```

```
[-8, -4, 0, 4, 8]
```

```
# create a new list only with the positive values of a list
```

```
>> [x for x in vec if x >= 0]
```

```
[0, 2, 4]
```

```
# create a new list by calling a function on every element of a list
```

```
>> [abs(x) for x in vec]
```

```
[4, 2, 0, 2, 4]
```

Examples

create a new list by calling a method on every element in a list

```
>> fruits = [' mela ', ' mirtillo ', ' pompelmo rosa ']
```

```
>> [fruit.strip() for fruit in fruits]
```

```
['mela', 'mirtillo', 'pompelmo rosa']
```

create a new list of tuples (element, its square value)

for the first 6 non-negative integers

```
>> [(x, x**2) for x in range(6)]
```

```
[(0, 0), (1, 1), (2, 4), (3, 9), (4, 16), (5, 25)]
```

remember tuple values go into (,)

```
>> [x, x**2 for x in range(6)]
```

SyntaxError: invalid syntax

Nested List Comprehension

- It is possible to apply list comprehension over a list coming from a list comprehension

```
>> l = ['*'+s+'*' for s in [c.upper() for c in 'abcd']]  
>> print(l)
```

```
['*A*', '*B*', '*C*', '*D*']
```

- The initial expression of a list comprehension can be a list comprehension

```
>> l = [[c*2 for c in word] for word in ['ab', 'cd']]  
>> print(l)
```

```
[['aa', 'bb'], ['cc', 'dd']]
```


Exercises

- **Ex 10.1** Print the list of numbers that are multiples of 7 and smaller than or equal to 100.
- **Ex 10.2** Print the list of numbers that contain the digit 3 and are less than or equal to 100.
- **Ex 10.3** Given a string, print the list of words contained in the string that have less than 4 letters.
- **Ex 10.4** Given a string, create a list of integers that represent the lengths of each word, but only if the word is not “the”.

Exercises

- **Ex 10.5** Print a list of three integers taken from a user via **input()**.
- **Ex 10.6** Write a function that flattens a given matrix and returns the elements as a list.

E.g.,

```
>> flatten([[1,2,3], [4,5,6], [7,8,9]])  
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Exercises

- **Ex 10.7** What is the result of the following command?

```
[ (x, y, z) for x in range(0,2) for y in range(0,3) if x != y for z in range(0,3) if z < x ]
```

- **Ex 10.8** Print the list of all the numbers from 1 to 1000 that are divisible by all numbers between 2 and 6.
- **Ex 10.9** Print the list of multiplication results from 0x0 to 10x10, that is, a matrix in which each row is the times table of n (from 0 to 10).

Generators

- Comprehension using round brackets does not create tuples, but generators that generate a sequence of values over time
 - The entire sequence is not created and allocated in the memory
 - A generator returns one value at a time and waits until the next value is called

E.g., `>> my_range = (x for x in range(10) if x%2 == 0)`
`>> type(my_range)`

`<class 'generator'>`

```
for x in my_range:
    print(x)
```

0

2

...

98

Generators

- Comprehension using round brackets does not create tuples, but generators that generate a sequence of values over time
 - The entire sequence is not created and allocated in the memory
 - A generator returns one value at a time and waits until the next value is called
- Generators can also be used with functions that take sequences or iterators as parameters

E.g., to calculate $\sum_{i=0}^{10} i$

```
>> sum(i for i in range(11))
```

55

Exercises

- **Ex 10.10** Write a function that multiplies the elements of a sequence of numbers (using a for loop on the elements). Call this function with a generator that generates a sequence of odd numbers less than 10.
- **Ex 10.11** Write a function that verifies that the sum of the numbers from 1 to a given non-negative number n is equal to $n(n+1)/2$.

Tuple Comprehension

- For tuple comprehension, we need to use `tuple()`

E.g., `>> tuple(x for x in range(100) if x%2 == 0)`

```
(0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24,
26, 28, 30, 32, 34, 36, 38, 40, 42, 44, 46,
48, 50, 52, 54, 56, 58, 60, 62, 64, 66, 68,
70, 72, 74, 76, 78, 80, 82, 84, 86, 88, 90,
92, 94, 96, 98)
```

Exercise 10.12

- Write a function that given a tuple of ranges, returns a tuple which contains all the values included in the given ranges.

E.g., given `(range(1,5), range(10,12), range(20,31))`, the function returns the tuple `(1,2,3,4,10,11,20,21,22,23,24,25,26,27,28,29,30)`

Dictionary Comprehension

- If we use the braces `{}` and the expression is a `key:value` pair then we build a dictionary

E.g.,

```
>> {x: x**2 for x in (2, 4, 6)}
```

```
{2: 4, 4: 16, 6: 36}
```

```
>> l = ['students', 'are', 'too', 'much', 'quiet']
```

```
>> {word:len(word) for word in l}
```

```
{'students': 8, 'are': 3, 'too': 3, 'much': 4, 'quiet': 5}
```

```
>> {c:c.upper() for c in 'characters'}
```

```
{'c': 'C', 'h': 'H', 'a': 'A', 'r': 'R', 't': 'T', 'e': 'E', 's': 'S'}
```


Exercise 10.13

- Write a function that inverts a dictionary:
 - each distinct value becomes a key;
 - for each inverted dictionary key, the value will be a list of the original keys associated with that value.

E.g.,

```
>> d = {'a':2, 'b':2, 'c':3, 'd':4, 'e':3, 'f':0, 'g':3, 'h':2}  
>> reverse(d)
```

```
{2: ['a', 'b', 'h'], 3: ['c', 'e', 'g'], 4: ['d'], 0: ['f']}
```